



IUS
INSTITUT
UNIVERSITAIRE
DES SCIENCES

Faculté des Sciences et des Technologies

Laboratoire N° 5 _ Modélisation Mathématique

Niveau : Licence III

Etudiant : Donsam Jean Gabard NOEL

Professeur : Ismaël SAINT ARMOUR

Date : 09 Mai 2026

TD1 - Analyse du réseau routier de Cap-Haïtien avec OpenStreetMap

```
C:\> Users > Downloads > TD1_cap-haïtien_3.py > ...  
1 # --- TD1 - Analyse du réseau routier de Cap-Haïtien ---  
2  
3 import networkx as nx  
4 import matplotlib.pyplot as plt  
5 import matplotlib.patches as mpatches  
6 import math  
7  
8 try:  
9     import osmnx as ox  
10 except ImportError:  
11     print("OSMnx non installé. Exécutez : pip install osmnx")  
12     raise  
13  
14 # -- ÉTAPE 1 - Charger la carte de Cap-Haïtien --  
15 print("-" * 60)  
16 print("    TD1 - Réseau routier de Cap-Haïtien (OSMnx)")  
17 print("-" * 60)  
18  
19 CAP_GPS = (19.7576, -72.2022)  
20 RAYON_M = 3000  
21  
22 print(f"\n[1] Chargement du graphe routier de Cap-Haïtien...")  
23 print(f"        Méthode : graph_from_point "  
24       f"(lat={CAP_GPS[0]}, lon={CAP_GPS[1]}, rayon={RAYON_M} m)")  
25  
26 # OSMnx retourne un MultiDiGraph  
27 G_raw = ox.graph_from_point(CAP_GPS, dist=RAYON_M,  
28                             network_type="drive", retain_all=False)  
29
```

```

30 print(f"          Sommets (intersections) : {len(G_raw.nodes),}")
31 print(f"          Arêtes (tronçons)       : {len(G_raw.edges),}")
32
33 # -- ÉTAPE 2 – Nettoyer le graphe --
34 print("\n[2] Nettoyage du graphe...")
35
36 # Plus grande composante fortement connexe (sur MultiDiGraph)
37 largest_cc = max(nx.strongly_connected_components(G_raw), key=len)
38 G = G_raw.subgraph(largest_cc).copy() # reste un MultiDiGraph
39
40 print(f"          Plus grande composante fortement connexe :")
41 print(f"          Sommets : {len(G.nodes),} | Arêtes : {len(G.edges),}")
42
43 # Conversion en graphe SIMPLE (DiGraph) pour les métriques
44 G_simple = nx.DiGraph(G) # DiGraph simple (orienté)
45 G_simple_u = G_simple.to_undirected() # Graph simple (non-orienté)
46
47 # -- ÉTAPE 3 – Calculer les métriques --
48 print("\n[3] Métriques du réseau :")
49
50 # Densité (sur graphe simple non-orienté)
51 densite = nx.density(G_simple_u)
52 print(f"          • Densité du réseau : {densite:.6f}")
53
54 # Degré moyen et max (sur le MultiDiGraph originel)
55 degres = dict(G.degree())
56 deg_moy = sum(degres.values()) / len(degres)
57 deg_max = max(degres.values())
58 print(f"          • Degré moyen : {deg_moy:.2f}")

```

```

61 # Composantes connexes
62 nb_comp = nx.number_connected_components(G_simple_u)
63 print(f"    Composantes connexes      : {nb_comp}")
64
65 # Longueur totale du réseau
66 longueur_km = sum(
67     d.get("length", 0) for _, _, d in G.edges(data=True)
68 ) / 1000
69 print(f"    Longueur totale du réseau   : {longueur_km:.1f} km")
70
71 # Transitivity (≈ clustering moyen, compatible avec tous types de graphe)
72 transitivity = nx.transitivity(G_simple_u)
73 print(f"    Transitivité (clustering)    : {transitivity:.4f}")
74
75 # -- ÉTAPE 4 – Identifier les points critiques --
76 print("\n[4] Identification des points critiques...")
77 print("    (centralité d'intermédiarité, approximation k=200)")
78
79 k_sample = min(200, len(G_simple.nodes))
80 centrality = nx.betweenness_centrality(
81     G_simple, k=k_sample, normalized=True, weight="length"
82 )
83 top10 = sorted(centrality.items(), key=lambda x: -x[1][:10])
84
85 print(f"\n    Top 10 intersections stratégiques de Cap-Haïtien :")
86 print(f"    {'Rang':<5} {'Node ID':<14} {'Centralité':>12} "
87       f"{'Latitude':>11} {'Longitude':>12}")
88 print(f"    {'-'*58}")
89 for rang, (node, val) in enumerate(top10, 1):

```

```

90     d = G.nodes[node]
91     print(f"    {'rang':<5} {'str(node)':<14} {'val':>12.5f} "
92           f"{'d.get('y', 0)':>11.5f} {'d.get('x', 0)':>12.5f}")
93
94     top_node = top10[0][0]
95     d_top = G.nodes[top_node]
96     print(f"\n    Point le plus stratégique : Node {top_node}")
97     print(f"    Lat={d_top.get('y',0):.5f}, Lon={d_top.get('x',0):.5f}")
98     print(f"    Nœud critique pour trafic, sécurité et urbanisme.")
99
100 # -- ÉTAPE 5 – Visualisation avancée --
101 print("\n[5] Visualisation avancée du réseau...")
102
103 top10_set = {n for n, _ in top10}
104 node_colors = []
105 node_sizes = []
106 for node in G.nodes():
107     c = centrality.get(node, 0)
108     if node in top10_set:
109         node_colors.append("EF4444") # rouge : points critiques
110         node_sizes.append(60)
111     elif c > 0.001:
112         node_colors.append("F59E0B") # orange : centralité moyenne
113         node_sizes.append(12)
114     else:
115         node_colors.append("3B82F6") # bleu : nœuds ordinaires
116         node_sizes.append(4)
117

```

```

118     edge_colors = [
119         "#22C55E"
120         if d.get("highway") in ("primary", "secondary", "trunk", "motorway")
121         else "#2D3748"
122         for _, _, d in G.edges(data=True)
123     ]
124
125     fig, ax = ox.plot_graph(
126         G,
127         node_color=node_colors,
128         node_size=node_sizes,
129         edge_color=edge_colors,
130         edge_linewidth=0.6,
131         bgcolor="#060B18",
132         show=False,
133         close=False,
134         figsize=(14, 12),
135     )
136
137     legende = [
138         mpatches.Patch(color="#EF4444", label="Points critiques (Top 10)"),
139         mpatches.Patch(color="#F59E0B", label="Centralité élevée"),
140         mpatches.Patch(color="#3B82F6", label="Nœuds ordinaires"),
141         mpatches.Patch(color="#22C55E", label="Routes principales"),
142     ]
143     ax.legend(handles=legende, loc="lower left",
144             facecolor="#0F172A", edgecolor="#475569",
145             labelcolor="white", fontsize=10)
146     ax.set_title(

```

```

141         mpatches.Patch(color="#22C55E", label="Routes principales"),
142     ]
143     ax.legend(handles=legende, loc="lower left",
144             facecolor="#0F172A", edgecolor="#475569",
145             labelcolor="white", fontsize=10)
146     ax.set_title(
147         f"TD1 - Réseau routier de Cap-Haïtien\n"
148         f"{len(G.nodes):,} intersections | {len(G.edges):,} tronçons | "
149         f"{longueur_km:.0f} km de routes",
150         color="white", fontsize=13, pad=15,
151     )
152
153     plt.savefig("TD1_cap_haitien_analyse.png", dpi=150,
154             bbox_inches="tight", facecolor="#060B18")
155     print("    Graphique sauvegardé : TD1_cap_haitien_analyse.png")
156     plt.show()
157
158     print("\n" + "-" * 60)
159     print("-" * 60)
160

```

TD1 – Réseau routier de Cap-Haïtien (OSMnx)

[1] Chargement du graphe routier de Cap-Haïtien...

Méthode : graph_from_point (lat=19.7576, lon=-72.2022, rayon=3000 m)

Sommets (intersections) : 1,532

Arêtes (tronçons) : 4,290

[2] Nettoyage du graphe...

Plus grande composante fortement connexe :

Sommets : 1,532 | Arêtes : 4,290

[3] Métriques du réseau :

- Densité du réseau : 0.001856
- Degré moyen : 5.60
- Degré max (carrefour busy) : 10
- Composantes connexes : 1
- Longueur totale du réseau : 355.4 km
- Transitivity (clustering) : 0.0255

[4] Identification des points critiques...

(centralité d'intermédiarité, approximation k=200)

Top 10 intersections stratégiques de Cap-Haïtien :

Rang	Node ID	Centralité	Latitude	Longitude
1	614617135	0.38613	19.75116	-72.20488
2	620039877	0.37198	19.73957	-72.19036
3	619785560	0.37174	19.73945	-72.19018
4	331017396	0.36866	19.73928	-72.18999

Top 10 intersections stratégiques de Cap-Haïtien :

Rang	Node ID	Centralité	Latitude	Longitude
1	614617135	0.38613	19.75116	-72.20488
2	620039877	0.37198	19.73957	-72.19036
3	619785560	0.37174	19.73945	-72.19018
4	331017396	0.36866	19.73928	-72.18999
5	331017393	0.36733	19.73916	-72.18987
6	2272149724	0.36725	19.73888	-72.18965
7	614457867	0.33058	19.73841	-72.18933
8	616903091	0.33025	19.73825	-72.18921
9	2264979334	0.32987	19.73797	-72.18901
10	331017534	0.32375	19.74807	-72.20754

Point le plus stratégique : Node 614617135

Lat=19.75116, Lon=-72.20488

Nœud critique pour trafic, sécurité et urbanisme.

[5] Visualisation avancée du réseau...

Graphique sauvegardé : TD1_cap_haitien_analyse.png



TD2 - Optimisation du transport urbain à Port-au-Prince avec OSMnx

```

1
2 # ---- TD2 - Optimisation du transport urbain à Port-au-Prince ----
3
4 import networkx as nx
5 import matplotlib.pyplot as plt
6 import matplotlib.patches as mpatches
7 import math
8
9 try:
10     import osmnx as ox
11 except ImportError:
12     print(" OSMnx non installé. Exécutez : pip install osmnx")
13     raise
14
15 # Fonction utilitaire - nearest node SANS scikit-learn
16 def haversine(lat1, lon1, lat2, lon2):
17     """Distance en km entre deux points GPS."""
18     R = 6371
19     dlat = math.radians(lat2 - lat1)
20     dlon = math.radians(lon2 - lon1)
21     a = math.sin(dlat/2)**2 + \
22         math.cos(math.radians(lat1)) * math.cos(math.radians(lat2)) * \
23         math.sin(dlon/2)**2
24     return R * 2 * math.asin(math.sqrt(a))
25
26
27 def nearest_node(G, lat, lon):
28     """Trouve le nœud du graphe le plus proche d'un point GPS.
29     N'utilise PAS scikit-learn - fonctionne sans installation supplémentaire."""
30     # Utilise la fonction haversine pour trouver le nœud le plus proche
31     # du point GPS (lat, lon) en parcourant tous les nœuds du graphe G.
32     # Retourne le nœud le plus proche.
33     min_distance = float('inf')
34     min_node = None
35     for node in G.nodes():
36         distance = haversine(lat, lon, node[0], node[1])
37         if distance < min_distance:
38             min_distance = distance
39             min_node = node
40     return min_node
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

27 def nearest_node(G, lat, lon):
28     """Trouve le nœud du graphe le plus proche d'un point GPS.
29     N'utilise PAS scikit-learn – fonctionne sans installation supplémentaire."""
30     best_node, best_dist = None, float("inf")
31     for node, data in G.nodes(data=True):
32         d = haversine(lat, lon, data["y"], data["x"])
33         if d < best_dist:
34             best_dist, best_node = d, node
35     return best_node
36
37
38 # --- ÉTAPE 1 – Télécharger la carte de Port-au-Prince ---
39 print("-" * 60)
40 print(" TD2 – Transport urbain à Port-au-Prince (OSMnx)")
41 print("-" * 60)
42
43 PAP_GPS = (18.5489, -72.3388) # Champ-de-Mars – centre de P-au-P
44 RAYON_M = 4000
45
46 print(f"\n[1] Téléchargement du réseau routier de Port-au-Prince...")
47 print(f" Centre : lat={PAP_GPS[0]}, lon={PAP_GPS[1]}, rayon={RAYON_M} m")
48
49 G_raw = ox.graph_from_point(PAP_GPS, dist=RAYON_M,
50                             network_type="drive", retain_all=False)
51 print(f" Sommets : {len(G_raw.nodes):,} | Arêtes : {len(G_raw.edges):,}")
52
53 # --- ÉTAPE 2 – Nettoyer le graphe ---
54 print("\n[2] Nettoyage : extraction de la plus grande composante...")
55

```

```

53 # --- ÉTAPE 2 – Nettoyer le graphe ---
54 print("\n[2] Nettoyage : extraction de la plus grande composante...")
55
56 largest_cc = max(nx.strongly_connected_components(G_raw), key=len)
57 G = G_raw.subgraph(largest_cc).copy() # MultiDiGraph
58
59 # Graphe simple pour métriques et centralité
60 G_simple = nx.DiGraph(G)
61
62 print(f" Après nettoyage – Sommets : {len(G.nodes):,} "
63       f"| Arêtes : {len(G.edges):,}")
64
65 # --- ÉTAPE 3 – Calculer les trajets optimaux (Dijkstra) ---
66 print("\n[3] Calcul des trajets optimaux par Dijkstra...")
67
68 VITESSE_KMH = 25
69
70 # Paires origine-destination dans le périmètre du graphe
71 trajets_coords = [
72     {
73         "nom": "Champ-de-Mars → Delmas",
74         "orig": (18.5489, -72.3388),
75         "dest": (18.5614, -72.3074),
76     },
77     {
78         "nom": "Carrefour-Feuilles → Turgeau",
79         "orig": (18.5400, -72.3300),
80         "dest": (18.5350, -72.3150),
81     },

```

```

83     "nom": "Bel-Air → Bourdon",
84     "orig": (18.5500, -72.3400),
85     "dest": (18.5300, -72.3200),
86 },
87 ]
88
89 # Ajouter travel_time sur chaque arête du MultiDiGraph
90 for u, v, k, data in G.edges(data=True, keys=True):
91     length = data.get("length", 50)
92     try:
93         speed = float(str(data.get("maxspeed", VITESSE_KMH)).split())[0])
94     except Exception:
95         speed = VITESSE_KMH
96     data["travel_time"] = length / (speed * 1000 / 3600)
97
98 # Propager travel_time sur le DiGraph simple
99 for u, v, data in G_simple.edges(data=True):
100     if "travel_time" not in data:
101         data["travel_time"] = data.get("length", 50) / (VITESSE_KMH * 1000 / 3600)
102
103 resultats_trajets = []
104 for t in trajets_coords:
105     print(f"\n    Trajet : {t['nom']}")
106
107     # Nearest node SANS scikit-learn
108     orig_node = nearest_node(G, t["orig"][0], t["orig"][1])
109     dest_node = nearest_node(G, t["dest"][0], t["dest"][1])
110     print(f"        Nœud origine : {orig_node} | Nœud dest : {dest_node}")

```

```

110     print(f"        Nœud origine : {orig_node} | Nœud dest : {dest_node}")
111
112     if orig_node == dest_node:
113         print(f"        Même nœud, trajet ignoré.")
114         continue
115
116     try:
117         # Dijkstra sur le graphe simple (DiGraph) – plus stable
118         route_dist = nx.shortest_path(G_simple, orig_node, dest_node, weight="length")
119         dist_m = sum(
120             G_simple[u][v].get("length", 0)
121             for u, v in zip(route_dist[:-1], route_dist[1:])
122         )
123         dist_km = dist_m / 1000
124         temps_mn = dist_km / VITESSE_KMH * 60
125
126         route_time = nx.shortest_path(G_simple, orig_node, dest_node,
127                                     weight="travel_time")
128         temps_s = sum(
129             G_simple[u][v].get("travel_time", 0)
130             for u, v in zip(route_time[:-1], route_time[1:])
131         )
132
133         resultats_trajets.append({
134             "nom": t["nom"],
135             "dist_km": dist_km,
136             "temps_est": temps_mn,
137             "temps_reel": temps_s / 60,
138             "route_dist": route_dist,

```



```

139         "route_time": route_time,
140         "orig_node": orig_node,
141         "dest_node": dest_node,
142     })
143
144     print(f"        Dijkstra (distance) : {dist_km:.2f} km - "
145           f"~{temps_mn:.0f} min à {VITESSE_KMH} km/h")
146     print(f"        Dijkstra (temps)      : {temps_s/60:.1f} min")
147
148     except (nx.NetworkXNoPath, nx.NodeNotFound) as e:
149         print(f"        Aucun chemin : {e}")
150
151 # --- ÉTAPE 4 - Zones critiques ---
152 print("\n[4] Zones critiques (centralité d'intermédiarité)...")
153
154 k_sample = min(150, len(G_simple.nodes))
155 centrality = nx.betweenness_centrality(
156     G_simple, k=k_sample, normalized=True, weight="length"
157 )
158 top5 = sorted(centrality.items(), key=lambda x: -x[1])[:5]
159
160 print(f"\n    Top 5 zones critiques de Port-au-Prince :")
161 print(f"    {'Rang':<5} {'Centralité':>12} {'Latitude':>12} {'Longitude':>12}")
162 print(f"    {'-'*50}")
163 for rang, (node, val) in enumerate(top5, 1):
164     d = G.nodes[node]
165     print(f"    {rang:<5} {val:>12.5f} "
166           f"{d.get('y', 0):>12.5f} {d.get('x', 0):>12.5f}")
167

```

```

163 for rang, (node, val) in enumerate(top5, 1):
164     d = G.nodes[node]
165     print(f"    {rang:<5} {val:>12.5f} "
166           f"{d.get('y', 0):>12.5f} {d.get('x', 0):>12.5f}")
167
168 # --- ÉTAPE 5 - Visualisation ---
169 print("\n[5] Visualisation des trajets optimaux...")
170
171 if resultats_trajets:
172     r = resultats_trajets[0]
173
174     fig, axes = plt.subplots(1, 2, figsize=(16, 9))
175     fig.patch.set_facecolor("#060B18")
176     fig.suptitle(
177         f"TD2 - Port-au-Prince : {r['nom']}\n"
178         f"Optimisation du transport urbain (Dijkstra)",
179         color="white", fontsize=14, fontweight="bold",
180     )
181
182     configs = [
183         (axes[0], r["route_dist"], "#F59E0B",
184          f"Dijkstra (distance) : {r['dist_km']:.2f} km"),
185         (axes[1], r["route_time"], "#10B981",
186          f"Dijkstra (temps)      : {r['temps_reel']:.1f} min"),
187     ]
188
189     for ax, route, color, label in configs:
190         ox.plot_graph_route(
191             G, route,

```

```

191         G, route,
192         route_color=color, route_linewidth=4, route_alpha=0.9,
193         node_size=0, bgcolor="#060B18",
194         show=False, close=False, ax=ax,
195     )
196     orig_xy = (G.nodes[r["orig_node"]]["x"], G.nodes[r["orig_node"]]["y"])
197     dest_xy = (G.nodes[r["dest_node"]]["x"], G.nodes[r["dest_node"]]["y"])
198     ax.plot(*orig_xy, "o", color="#22C55E", markersize=14, zorder=5)
199     ax.plot(*dest_xy, "s", color="#EF4444", markersize=14, zorder=5)
200     ax.set_title(label, color="white", fontsize=12, pad=10)
201
202     legende = [
203         mpatches.Patch(color="#22C55E", label="Origine"),
204         mpatches.Patch(color="#EF4444", label="Destination"),
205     ]
206     fig.legend(handles=legende, loc="lower center", ncol=2,
207               facecolor="#0F172A", edgecolor="#475569",
208               labelcolor="white", fontsize=11)
209     plt.tight_layout()
210     plt.savefig("TD2_pap_trajets.png", dpi=150,
211               bbox_inches="tight", facecolor="#060B18")
212     print("    Graphique sauvegardé : TD2_pap_trajets.png")
213     plt.show()
214
215     # Tableau récapitulatif
216     print("\n    Tableau récapitulatif :")
217     print(f"    {'Trajet':<35} {'Dist':>8} {'T.dist':>10} {'T.opt':>10}")
218     print(f"    {'-'*66}")
219     for r2 in resultats_trajets:
220         print(f"    {r2['nom']:<35} {r2['dist_km']:>6.2f} km "

```

```

218         print(f"    {'-'*66}")
219         for r2 in resultats_trajets:
220             print(f"    {r2['nom']:<35} {r2['dist_km']:>6.2f} km "
221                   f"    {r2['temps_est']:>8.0f} min {r2['temps_reel']:>8.1f} min")
222     else:
223         print("    Aucun trajet disponible pour la visualisation.")
224
225     print("\n" + "-" * 60)
226     print("-" * 60)
227

```

TD2 – Transport urbain à Port-au-Prince (OSMnx)

[1] Téléchargement du réseau routier de Port-au-Prince...

Centre : lat=18.5489, lon=-72.3388, rayon=4000 m
Sommets : 3,764 | Arêtes : 9,696

[2] Nettoyage : extraction de la plus grande composante...

Après nettoyage – Sommets : 3,761 | Arêtes : 9,692

[3] Calcul des trajets optimaux par Dijkstra...

Trajet : Champ-de-Mars → Delmas

Nœud origine : 4053625231 | Nœud dest : 613083708
Dijkstra (distance) : 4.20 km – ~10 min à 25 km/h
Dijkstra (temps) : 10.1 min

Trajet : Carrefour-Feuilles → Turgeau

Nœud origine : 617528910 | Nœud dest : 892483147
Dijkstra (distance) : 2.65 km – ~6 min à 25 km/h
Dijkstra (temps) : 6.4 min

Trajet : Bel-Air → Bourdon

Nœud origine : 613617371 | Nœud dest : 612987776
Dijkstra (distance) : 3.52 km – ~8 min à 25 km/h
Dijkstra (temps) : 8.4 min

[4] Zones critiques (centralité d'intermédiarité)...

Top 5 zones critiques de Port-au-Prince :

Rang	Centralité	Latitude	Longitude
------	------------	----------	-----------

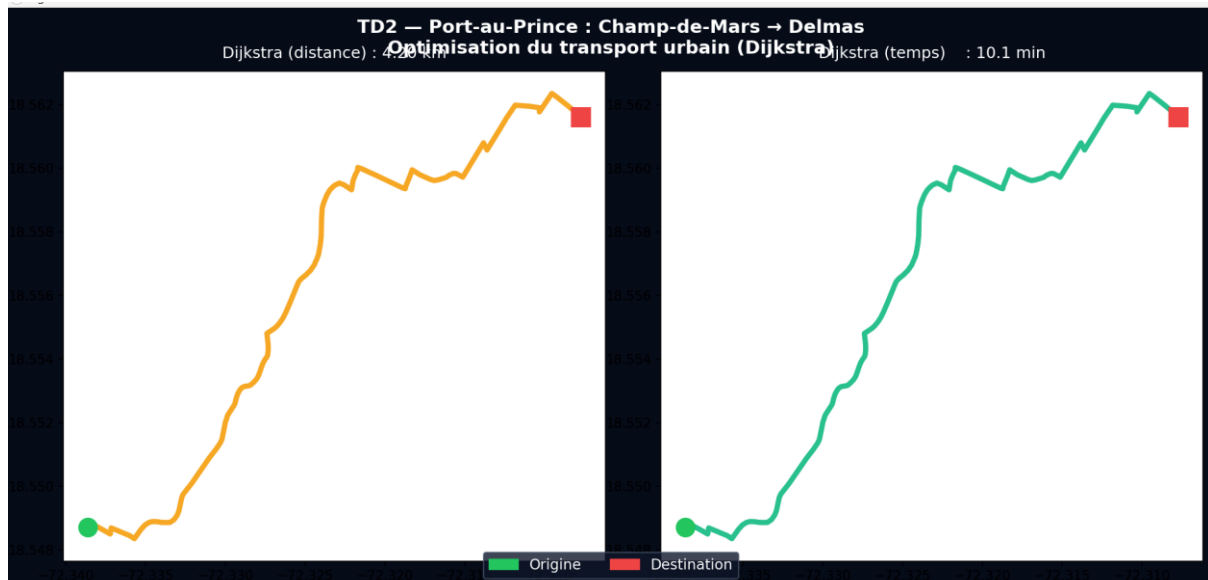
[4] Zones critiques (centralité d'intermédiarité)...

Top 5 zones critiques de Port-au-Prince :

Rang	Centralité	Latitude	Longitude
1	0.19567	18.53636	-72.34652
2	0.19379	18.53316	-72.34736
3	0.17808	18.53611	-72.34659
4	0.17313	18.54048	-72.33828
5	0.17229	18.54136	-72.33803

[5] Visualisation des trajets optimaux...

Graphique sauvegardé : TD2_pap_trajets.png



TD3 - Optimisation des transports (tap-tap)

```
2 # ---- TD3 - Optimisation des transports (tap-tap) ----
3
4 import networkx as nx
5 import matplotlib.pyplot as plt
6 import matplotlib.patches as mpatches
7 import math
8
9 # --- ÉTAPE 1 - Modéliser le réseau de transport tap-tap ---
10 print(" TD3 - Optimisation du réseau tap-tap (Port-au-Prince)")
11 print("-" * 60)
12
13 print("\n[1] Modélisation du réseau tap-tap...")
14
15 # Zones clés de Port-au-Prince avec coordonnées GPS réelles
16 zones_gps = {
17     "Champ-de-Mars": (18.5489, -72.3388),
18     "Delmas": (18.5614, -72.3074),
19     "Pétion-Ville": (18.5133, -72.2869),
20     "Tabarre": (18.5942, -72.2906),
21     "Carrefour": (18.5344, -72.3708),
22     "Cité Soleil": (18.5788, -72.3464),
23     "Croix-des-Bossales": (18.5459, -72.3465),
24     "Delmas 33": (18.5500, -72.3100),
25     "Kenscoff Rd": (18.4900, -72.2800),
26 }
27
28 def haversine(c1, c2):
29     """Distance réelle en km entre deux points GPS."""
30     R = 6371
```

```

28 def haversine(c1, c2):
29     """
30     """
31     lat1, lon1 = math.radians(c1[0]), math.radians(c1[1])
32     lat2, lon2 = math.radians(c2[0]), math.radians(c2[1])
33     a = math.sin((lat2-lat1)/2)**2 + \
34         math.cos(lat1)*math.cos(lat2)*math.sin((lon2-lon1)/2)**2
35     return R * 2 * math.asin(math.sqrt(a))
36
37 # Construction du graphe - lignes tap-tap existantes
38 # Chaque arête = une ligne tap-tap avec sa distance réelle
39 lignes_taptap = [
40     # (Zone A, Zone B, ligne)
41     ("Champ-de-Mars", "Delmas", "Ligne 1"),
42     ("Champ-de-Mars", "Carrefour", "Ligne 2"),
43     ("Champ-de-Mars", "Cité Soleil", "Ligne 3"),
44     ("Champ-de-Mars", "Croix-des-Bossales", "Ligne 4"),
45     ("Delmas", "Pétion-Ville", "Ligne 5"),
46     ("Delmas", "Tabarre", "Ligne 6"),
47     ("Delmas", "Delmas 33", "Ligne 7"),
48     ("Pétion-Ville", "Kenscoff Rd", "Ligne 8"),
49     ("Carrefour", "Croix-des-Bossales", "Ligne 9"),
50     ("Cité Soleil", "Tabarre", "Ligne 10"),
51     ("Cité Soleil", "Croix-des-Bossales", "Ligne 11"),
52     ("Delmas 33", "Pétion-Ville", "Ligne 12"),
53 ]
54
55 G = nx.Graph()
56 for z_a, z_b, ligne in lignes_taptap:
57     dist = haversine(zones_gps[z_a], zones_gps[z_b])
58     G.add_edge(z_a, z_b, weight=round(dist, 2), ligne=ligne)

```

```

56 for z_a, z_b, ligne in lignes_taptap:
57     dist = haversine(zones_gps[z_a], zones_gps[z_b])
58     G.add_edge(z_a, z_b, weight=round(dist, 2), ligne=ligne)
59
60 print(f"    Zones modélisées : {len(G.nodes)}")
61 print(f"    Lignes tap-tap : {len(G.edges)}")
62 print(f"\n    Lignes du réseau actuel :")
63 print(f"    {'Ligne':<12} {'Trajet':<45} {'Distance':>10}")
64 print(f"    {'-'*70}")
65 for u, v, d in sorted(G.edges(data=True), key=lambda e: e[2]["weight"]):
66     print(f"    {d['ligne']:<12} {u} ↔ {v:<30} {d['weight']:>8.2f} km")
67
68 dist_totale_actuelle = sum(d["weight"] for _, _, d in G.edges(data=True))
69 print(f"\n    Distance totale du réseau actuel : {dist_totale_actuelle:.2f} km")
70
71
72 # --- ÉTAPE 2 - Trouver les itinéraires optimaux ---
73
74 print("\n[2] Itinéraires optimaux (algorithme de Dijkstra) :")
75
76 VITESSE_KMH = 20 # tap-tap : ~20 km/h en ville
77
78 demandes = [
79     ("Cité Soleil", "Pétion-Ville"),
80     ("Carrefour", "Tabarre"),
81     ("Kenscoff Rd", "Cité Soleil"),
82     ("Tabarre", "Carrefour"),
83     ("Delmas", "Kenscoff Rd"),
84 ]

```

```

86 print(f"\n    Vitesse moyenne tap-tap supposée : {VITESSE_KMH} km/h")
87 print(f"    {'Origine':<22} {'Destination':<22} {'Chemin':<50} {'km':>6} {'min':>6}")
88 print(f"    {'-'*110}")
89
90 resultats_routes = []
91 for src, dst in demandes:
92     try:
93         chemin = nx.shortest_path(G, src, dst, weight="weight")
94         dist = nx.shortest_path_length(G, src, dst, weight="weight")
95         temps = dist / VITESSE_KMH * 60
96         chemin_str = " → ".join(chemin)
97         print(f"    {src:<22} {dst:<22} {chemin_str:<50} {dist:>6.2f} {temps:>6.1f}")
98         resultats_routes.append((src, dst, chemin, dist, temps))
99     except nx.NetworkXNoPath:
100         print(f"    {src:<22} {dst:<22} Aucun chemin trouvé")
101
102 # --- ÉTAPE 3 – Proposer une amélioration (MST = réseau optimisé) ---
103
104 print("\n[3] Amélioration du réseau – Arbre Couvrant Minimal (Kruskal) :")
105 print("    L'MST donne le réseau minimum qui connecte toutes les zones.")
106
107 T = nx.minimum_spanning_tree(G, algorithm="kruskal", weight="weight")
108 dist_mst = sum(d["weight"] for _, _, d in T.edges(data=True))
109
110 print(f"\n    Réseau actuel    : {len(G.edges)} lignes, {dist_totale_actuelle:.2f} km totaux")
111 print(f"    Réseau optimisé  : {len(T.edges)} lignes, {dist_mst:.2f} km totaux")
112 print(f"    Économie        : {dist_totale_actuelle - dist_mst:.2f} km "
113       f"    f"(-(1 - dist_mst/dist_totale_actuelle)*100:.1f)%)")
114

```

```

115 print(f"\n    Lignes conservées dans le réseau optimisé (MST) :")
116 print(f"    {'Trajet':<45} {'Distance':>10}")
117 print(f"    {'-'*58}")
118 for u, v, d in sorted(T.edges(data=True), key=lambda e: e[2]["weight"]):
119     print(f"    {u} ↔ {v:<30} {d['weight']:>8.2f} km")
120
121 lignes_supprimees = set(G.edges()) - set(T.edges()) - {(v,u) for u,v in T.edges()}
122 print(f"\n    Lignes redondantes à supprimer ou fusionner :")
123 for u, v in G.edges():
124     if (u, v) not in T.edges() and (v, u) not in T.edges():
125         d = G[u][v]["weight"]
126         ligne = G[u][v]["ligne"]
127         print(f"    X {ligne:<12} {u} ↔ {v}    ({d:.2f} km)")
128
129 # Visualisation
130 pos = {z: (coords[1], coords[0]) for z, coords in zones_gps.items()}
131
132 fig, axes = plt.subplots(1, 2, figsize=(18, 8))
133 fig.patch.set_facecolor(□"#060B18")
134 fig.suptitle("TD3 – Optimisation du réseau tap-tap de Port-au-Prince",
135             color="white", fontsize=15, fontweight="bold")
136
137 mst_edges_set = set(T.edges()) | {(v, u) for u, v in T.edges()}
138
139 def dessiner_reseau(ax, G_draw, titre, mst_edges_set=None):
140     ax.set_facecolor(□"#060B18")
141
142     if mst_edges_set is not None:
143         edge_colors = [□"#10B981" if ((u,v) in mst_edges_set) else □"#EF4444"

```

```

139 def dessiner_reseau(ax, G_draw, titre, mst_edges_set=None):
140     for u,v in G_draw.edges():
141         edge_widths = [2.5 if ((u,v) in mst_edges_set) else 1.2
142                         for u,v in G_draw.edges()]
143     else:
144         edge_colors = [ "#6366F1" ] * len(G_draw.edges())
145         edge_widths = [2.0] * len(G_draw.edges())
146
147     nx.draw_networkx_nodes(G_draw, pos, ax=ax, node_color="#F59E0B",
148                           node_size=700, edgecolors="white", linewidths=1.5)
149     nx.draw_networkx_labels(G_draw, pos, ax=ax, font_color="#0F172A",
150                            font_size=7.5, font_weight="bold")
151     nx.draw_networkx_edges(G_draw, pos, ax=ax, edge_color=edge_colors,
152                           width=edge_widths)
153     edge_labels = {(u,v): f"{d['weight']:.1f} km" for u,v,d in G_draw.edges(data=True)}
154     nx.draw_networkx_edge_labels(G_draw, pos, edge_labels=edge_labels, ax=ax,
155                                 font_size=6.5, font_color="#CBD5E1")
156     ax.set_title(titre, color="white", fontsize=12, pad=12)
157     ax.axis("off")
158
159 # Panneau gauche : réseau actuel avec MST mis en avant
160 dessiner_reseau(axes[0], G,
161                 f"Réseau actuel ({len(G.edges)} lignes - {dist_totale_actuelle:.1f} km)",
162                 mst_edges_set)
163
164 patch_garde = mpatches.Patch(color="#10B981", label="Gardée dans MST")
165 patch_suppr = mpatches.Patch(color="#EF4444", label="Redondante (à optimiser)")
166 axes[0].legend(handles=[patch_garde, patch_suppr], loc="lower left",
167                facecolor="#0F172A", edgecolor="#475569", labelcolor="white", fontsize=9)

```

```

170 axes[0].legend(handles=[patch_garde, patch_suppr], loc="lower left",
171                facecolor="#0F172A", edgecolor="#475569", labelcolor="white", fontsize=9)
172
173 # Panneau droit : réseau optimisé (MST uniquement)
174 dessiner_reseau(axes[1], T,
175                 f"Réseau optimisé - MST ({len(T.edges)} lignes - {dist_mst:.1f} km)\n"
176                 f"Économie : {dist_totale_actuelle - dist_mst:.1f} km")
177
178 plt.tight_layout()
179 plt.savefig("TD3_taptap_optimisation.png", dpi=150, bbox_inches="tight",
180           facecolor="#FFFFFF")
181 print("\n Graphique sauvegardé : TD3_taptap_optimisation.png")
182 fig.patch.set_facecolor('white')
183 plt.show()
184
185 print("\n" + "-" * 60)
186 print("-" * 60)
187

```

TD3 – Optimisation du réseau tap-tap (Port-au-Prince)

[1] Modélisation du réseau tap-tap...

Zones modélisées : 9
Lignes tap-tap : 12

Lignes du réseau actuel :

Ligne	Trajet	Distance
Ligne 4	Champ-de-Mars ↔ Croix-des-Bossales	0.88 km
Ligne 7	Delmas ↔ Delmas 33	1.30 km
Ligne 8	Pétion-Ville ↔ Kenscoff Rd	2.69 km
Ligne 9	Carrefour ↔ Croix-des-Bossales	2.86 km
Ligne 3	Champ-de-Mars ↔ Cité Soleil	3.42 km
Ligne 1	Champ-de-Mars ↔ Delmas	3.59 km
Ligne 11	Cité Soleil ↔ Croix-des-Bossales	3.66 km
Ligne 2	Champ-de-Mars ↔ Carrefour	3.74 km
Ligne 6	Delmas ↔ Tabarre	4.05 km
Ligne 12	Pétion-Ville ↔ Delmas 33	4.75 km
Ligne 5	Delmas ↔ Pétion-Ville	5.77 km
Ligne 10	Cité Soleil ↔ Tabarre	6.13 km

Distance totale du réseau actuel : 42.84 km

[2] Itinéraires optimaux (algorithme de Dijkstra) :

Vitesse moyenne tap-tap supposée : 20 km/h

Origine	Destination	Chemin	km	min
Cité Soleil	Pétion-Ville	Cité Soleil → Champ-de-Mars → Delmas → Pétion-Ville	12.78	38.3
Carrefour	Tabarre	Carrefour → Croix-des-Bossales → Champ-de-Mars → Delmas → Tabarre	11.38	34.1

Carrefour	Tabarre	Carrefour → Croix-des-Bossales → Champ-de-Mars → Delmas → Tabarre	11.38	34.1
Kenscoff Rd	Cité Soleil	Kenscoff Rd → Pétion-Ville → Delmas → Champ-de-Mars → Cité Soleil	15.47	46.4
Tabarre	Carrefour	Tabarre → Delmas → Champ-de-Mars → Croix-des-Bossales → Carrefour	11.38	34.1
Delmas	Kenscoff Rd	Delmas → Pétion-Ville → Kenscoff Rd	8.46	25.4

[3] Amélioration du réseau – Arbre Couvrant Minimal (Kruskal) :
L'MST donne le réseau minimum qui connecte toutes les zones.

Réseau actuel : 12 lignes, 42.84 km totaux
Réseau optimisé : 8 lignes, 23.54 km totaux
Économie : 19.30 km (-45.1%)

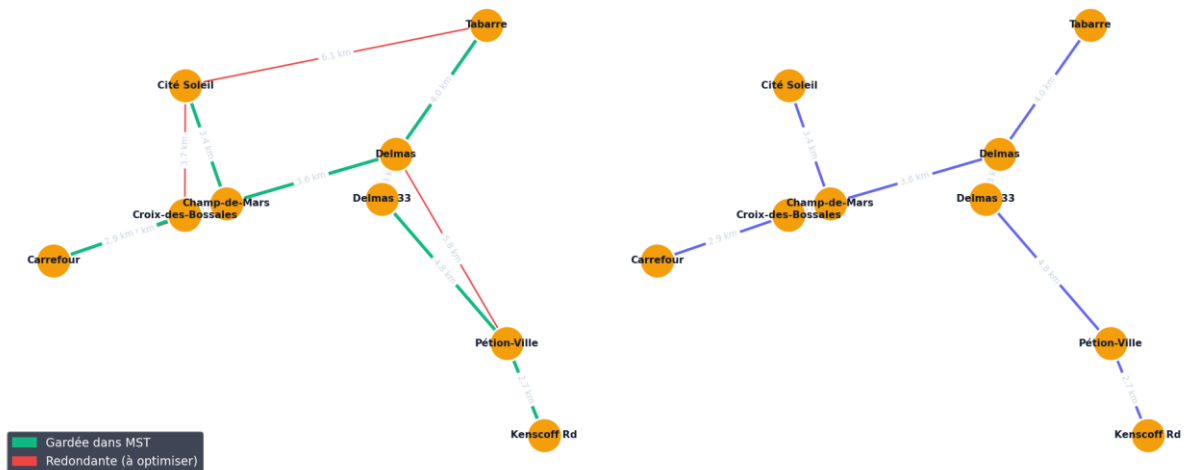
Lignes conservées dans le réseau optimisé (MST) :

Trajet	Distance
Champ-de-Mars ↔ Croix-des-Bossales	0.88 km
Delmas ↔ Delmas 33	1.30 km
Pétion-Ville ↔ Kenscoff Rd	2.69 km
Carrefour ↔ Croix-des-Bossales	2.86 km
Champ-de-Mars ↔ Cité Soleil	3.42 km
Champ-de-Mars ↔ Delmas	3.59 km
Delmas ↔ Tabarre	4.05 km
Pétion-Ville ↔ Delmas 33	4.75 km

Lignes redondantes à supprimer ou fusionner :

X Ligne 2 Champ-de-Mars ↔ Carrefour (3.74 km)
X Ligne 5 Delmas ↔ Pétion-Ville (5.77 km)
X Ligne 10 Cité Soleil ↔ Tabarre (6.13 km)
X Ligne 11 Cité Soleil ↔ Croix-des-Bossales (3.66 km)

Graphique sauvegardé : TD3_taptap_optimisation.png



TD 4 - Comparaison entre villes

```
1  # ---- TD4 - Comparaison entre villes d'Haïti ----
2
3  import networkx as nx
4  import matplotlib.pyplot as plt
5  import math
6  import warnings
7  warnings.filterwarnings("ignore")
8
9  try:
10     import osmnx as ox
11 except ImportError:
12     print("⚠ OSMnx non installé. Exécutez : pip install osmnx")
13     raise
14
15 print("-" * 65)
16 print(" TD4 - Comparaison des réseaux routiers des villes haïtiennes")
17 print("-" * 65)
18
19 # Coordonnées GPS de chaque ville + rayon
20 VILLES = {
21     "Port-au-Prince": (18.5489, -72.3388),
22     "Cap-Haïtien": (19.7576, -72.2022),
23     "Les Cayes": (18.1937, -73.7492),
24     "Jacmel": (18.2341, -72.5348),
25     "Port-de-Paix": (19.9318, -72.8299),
26 }
27 RAYON_M = 2500
```

```
29 # - Calcul des métriques -
30 def calculer_metriques(nom, gps, rayon):
31     """Télécharge et analyse le réseau routier d'une ville."""
32
33     # MultiDiGraph OSMnx
34     G_raw = ox.graph_from_point(gps, dist=rayon,
35                                network_type="drive", retain_all=False)
36
37     # Nettoyage : plus grande composante fortement connexe
38     largest = max(nx.strongly_connected_components(G_raw), key=len)
39     G = G_raw.subgraph(largest).copy()
40
41     # - Conversion en graphe SIMPLE pour les métriques -
42
43     G_simple = nx.DiGraph(G) # DiGraph simple orienté
44     G_simple_u = G_simple.to_undirected() # Graph simple non-orienté
45
46     nb_sommets = len(G.nodes)
47     nb_aretes = len(G.edges)
48
49     # Critère 1 - Densité (E / V sur graphe simple)
50     densite = nx.density(G_simple_u)
51
52     # Critère 2 - Connectivité
53     G_raw_simple_u = nx.Graph(G_raw)
54     nb_comp = nx.number_connected_components(G_raw_simple_u)
```

```
56     # Critère 3 - Accessibilité
57     degre_moy = sum(dict(G.degree()).values()) / nb_sommets
58     # transitivity = fraction de triplets fermés
59     transitivity = nx.transitivity(G_simple_u)
60
61     # Longueur totale
62     long_km = sum(
63         d.get("length", 0) for _, _, d in G.edges(data=True)
64     ) / 1000
65
66     # Centralité max (approximation)
67     k = min(80, nb_sommets)
68     centr = nx.betweenness_centrality(G_simple, k=k, normalized=True)
69     max_c = max(centr.values()) if centr else 0
70
71     return {
72         "Sommets": nb_sommets,
73         "Arêtes": nb_aretes,
74         "Longueur (km)": round(long_km, 1),
75         "Densité": round(densite, 6),
76         "Composantes": nb_comp,
77         "Degré moyen": round(degre_moy, 2),
78         "Transitivité": round(transitivity, 4),
79         "Max centralité": round(max_c, 5),
80     }
```



```

82  resultats = {}
83  for nom, gps in VILLES.items():
84      print(f"\n Changement de {nom} (lat={gps[0]}, lon={gps[1]})...")
85      try:
86          m = calculer_metriques(nom, gps, RAYON_M)
87          resultats[nom] = m
88          print(f"          {m['Sommets']:,} sommets | {m['Arêtes']:,} arêtes | "
89                f"          {m['Longueur (km)']:,} km | densité={m['Densité']:.6f}")
90      except Exception as e:
91          print(f"          Erreur pour {nom} : {e}")
92
93  # - Tableau comparatif -
94  print("\n\n" + "-" * 72)
95  print("  TABLEAU COMPARATIF")
96  print("-" * 72)
97
98  colonnes = ["Sommets", "Arêtes", "Longueur (km)", "Densité",
99             "Composantes", "Degré moyen", "Transitivité", "Max centralité"]
100
101  entete = f"  {'Ville':<20}" + "".join(f"{'c':>16}" for c in colonnes)
102  print(f"\n{entete}")
103  print(f"  {'-' * (20 + 16 * len(colonnes))}")
104  for ville, m in resultats.items():
105      ligne = f"  {ville:<20}" + "".join(f"{'str(m[c]):>16}" for c in colonnes)
106      print(ligne)

```

```

108  # - Interprétation -
109  print("\n\n" + "-" * 72)
110  print("  INTERPRÉTATION DES RÉSULTATS")
111  print("-" * 72)
112
113  if resultats:
114      ville_grande = max(resultats, key=lambda v: resultats[v]["Sommets"])
115      ville_dense = max(resultats, key=lambda v: resultats[v]["Densité"])
116      ville_longue = max(resultats, key=lambda v: resultats[v]["Longueur (km)"])
117      ville_transit = max(resultats, key=lambda v: resultats[v]["Transitivité"])
118      ville_centre = max(resultats, key=lambda v: resultats[v]["Max centralité"])
119
120      print(f"""
121      Réseau le plus étendu          : {ville_grande}
122      | Plus grand nombre d'intersections dans le périmètre analysé.
123
124      Réseau le plus dense           : {ville_dense}
125      | Maillage serré = plus d'alternatives de trajet.
126
127      Ville avec + de km de routes   : {ville_longue}
128
129      Meilleure transitivité locale  : {ville_transit}
130      | Quartiers bien maillés, bonne accessibilité locale.
131
132      Point stratégique le + fort    : {ville_centre}
133      | Priorité pour aménagements et sécurité routière.
134      """)

```

```

135      for ville, m in resultats.items():
136          etat = "bien connecté " if m["Composantes"] == 1 \
137                else f"fragmenté ({m['Composantes']} composantes) "
138          print(f"    • {ville:<20} → {etat}")
139
140  # - Visualisation -
141  print("\n\n  Génération des graphiques comparatifs...")
142
143  if not resultats:
144      print("    Aucun résultat disponible.")
145  else:
146      noms = list(resultats.keys())
147      short = [
148          n.replace("Port-au-Prince", "PaP")
149          .replace("Cap-Haïtien", "Cap-H.")
150          .replace("Port-de-Paix", "PdP")
151          for n in noms
152      ]
153
154      metriques_graph = [
155          ("Sommets", "#3B82F6", "Sommets (intersections)"),
156          ("Longueur (km)", "#10B981", "Longueur du réseau (km)"),
157          ("Densité", "#F59E0B", "Densité (E/V)"),
158          ("Degré moyen", "#85CF6", "Degré moyen"),
159          ("Transitivité", "#EF4444", "Transitivité (= clustering)"),
160          ("Max centralité", "#EC4899", "Centralité max (point stratégique)"),
161      ]

```

```

163 fig, axes = plt.subplots(2, 3, figsize=(18, 10))
164 fig.patch.set_facecolor("#060B18")
165 fig.suptitle(
166     "TD4 - Comparaison des réseaux routiers des villes haïtiennes\n"
167     f"(rayon d'analyse : {RAYON_M/1000:.1f} km par ville)",
168     color="white", fontsize=14, fontweight="bold",
169 )
170
171 for ax, (col, couleur, titre) in zip(axes.flat, metriques_graph):
172     valeurs = [resultats[v][col] for v in noms]
173     bars = ax.bar(short, valeurs, color=couleur,
174                  edgecolor="white", linewidth=0.8, width=0.55)
175     ax.set_facecolor("#0F172A")
176     ax.tick_params(colors="white", labelsize=9)
177     ax.spines[:].set_color("#374151")
178     ax.set_title(titre, color="white", fontsize=10, pad=8)
179     ax.set_xlabel("Ville", color="#94A3B8", fontsize=9)
180     for bar, val in zip(bars, valeurs):
181         ax.text(
182             bar.get_x() + bar.get_width() / 2,
183             bar.get_height() + max(valeurs) * 0.01,
184             f"{val}", ha="center", color="white",
185             fontsize=8, fontweight="bold",
186         )
187
188 plt.tight_layout()
189 plt.savefig("TD4_comparaison_villes.png", dpi=150,
190           bbox_inches="tight", facecolor="#060B18")
191 print("    Graphique sauvegardé : TD4_comparaison_villes.png")
192 plt.show()

```

TD4 - Comparaison des réseaux routiers des villes haïtiennes

Chargement de Port-au-Prince (lat=18.5489, lon=-72.3388)...

1,930 sommets | 5,110 arêtes | 510.1 km | densité=0.001445

Chargement de Cap-Haïtien (lat=19.7576, lon=-72.2022)...

1,166 sommets | 3,340 arêtes | 266.1 km | densité=0.002506

Chargement de Les Cayes (lat=18.1937, lon=-73.7492)...

698 sommets | 1,729 arêtes | 203.4 km | densité=0.003609

Chargement de Jacmel (lat=18.2341, lon=-72.5348)...

782 sommets | 2,016 arêtes | 250.4 km | densité=0.003291

Chargement de Port-de-Paix (lat=19.9318, lon=-72.8299)...

804 sommets | 2,044 arêtes | 210.6 km | densité=0.003178

TABLEAU COMPARATIF

Ville	Sommets	Arêtes	Longueur (km)	Densité	Composantes	Degré moyen	Transitivité	Max centralité
Port-au-Prince	1930	5110	510.1	0.001445	1	5.3	0.0503	0.19931
Cap-Haïtien	1166	3340	266.1	0.002506	1	5.73	0.0231	0.34508
Les Cayes	698	1729	203.4	0.003609	1	4.95	0.0348	0.36742
Jacmel	782	2016	250.4	0.003291	1	5.16	0.0405	0.2374
Port-de-Paix	804	2044	210.6	0.003178	1	5.08	0.0396	0.36293

INTERPRÉTATION DES RÉSULTATS

Réseau le plus étendu : Port-au-Prince
Plus grand nombre d'intersections dans le périmètre analysé.

Réseau le plus dense : Les Cayes
Maillage serré = plus d'alternatives de trajet.

Ville avec + de km de routes : Port-au-Prince

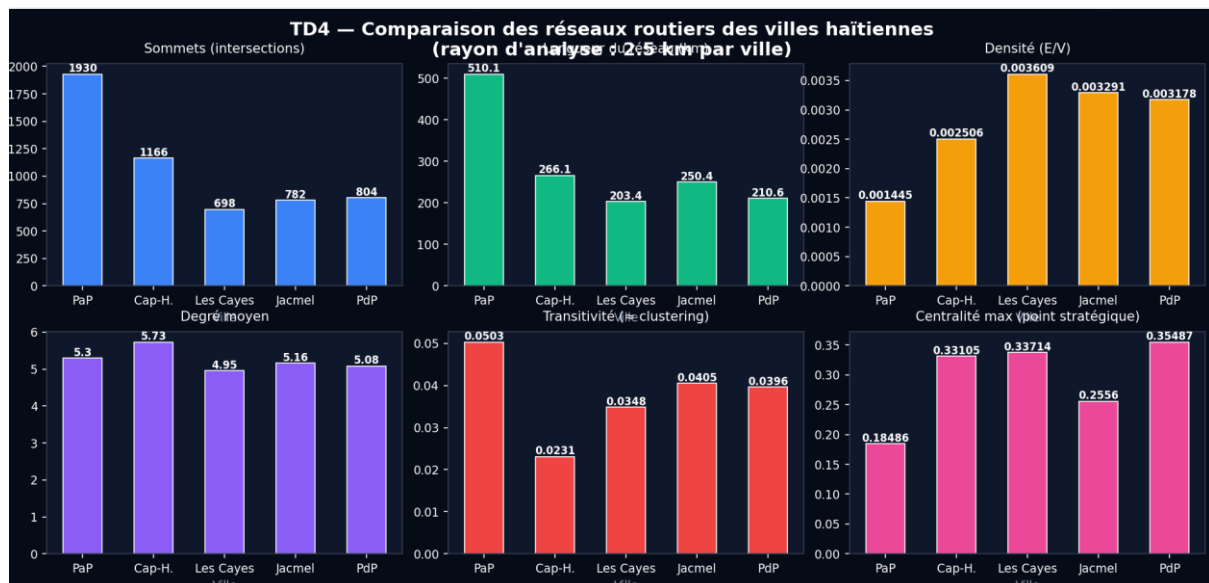
Meilleure transitivity locale : Port-au-Prince
Quartiers bien maillés, bonne accessibilité locale.

Point stratégique le + fort : Les Cayes
Priorité pour aménagements et sécurité routière.

- Port-au-Prince → bien connecté
- Cap-Haïtien → bien connecté
- Les Cayes → bien connecté
- Jacmel → bien connecté
- Port-de-Paix → bien connecté

Génération des graphiques comparatifs...

Graphique sauvegardé : TD4_comparaison_villes.png



TD5 -Optimisation de livraison

```
1 # --- TD5 - Optimisation de livraison ---
2
3 import networkx as nx
4 import matplotlib.pyplot as plt
5 import matplotlib.patches as mpatches
6 import math
7 import itertools
8 import random
9
10 random.seed(42)
11
12 # --- ÉTAPE 1 - Définir les points de livraison ---
13 print("-" * 65)
14 print(" TD5 - Optimisation de livraison (Port-au-Prince)")
15 print("-" * 65)
16
17 print("\n[1] Définition des points de livraison...")
18
19 # Points de livraison réels à Port-au-Prince (lat, lon)
20 POINTS = {
21     "Dépôt (Delmas 18)": (18.5614, -72.3182),
22     "Client 1 (Pétion-V.)": (18.5133, -72.2869),
23     "Client 2 (Tabarre)": (18.5942, -72.2906),
24     "Client 3 (Carrefour)": (18.5344, -72.3708),
25     "Client 4 (Cité Sol.)": (18.5788, -72.3464),
26     "Client 5 (Delmas 33)": (18.5500, -72.3100),
27     "Client 6 (Croix-B.)": (18.5459, -72.3465),
28 }
29
```

```

30 NOMS = list(POINTS.keys())
31 COORDS = list(POINTS.values())
32 N = len(NOMS)
33 DEPOT = 0 # index du dépôt
34
35 def haversine(c1, c2):
36     """Distance réelle en km entre deux coordonnées GPS."""
37     R = 6371
38     lat1, lon1 = math.radians(c1[0]), math.radians(c1[1])
39     lat2, lon2 = math.radians(c2[0]), math.radians(c2[1])
40     a = math.sin((lat2-lat1)/2)**2 + \
41         math.cos(lat1)*math.cos(lat2)*math.sin((lon2-lon1)/2)**2
42     return R * 2 * math.asin(math.sqrt(a))
43
44 # Matrice de distances
45 D = [[haversine(COORDS[i], COORDS[j]) for j in range(N)] for i in range(N)]
46
47 print(f"      {N} points définis (1 dépôt + {N-1} clients)")
48 print(f"\n      Points de livraison :")
49 for i, (nom, (lat, lon)) in enumerate(POINTS.items()):
50     role = " DÉPÔT" if i == DEPOT else f" Client {i}"
51     print(f"      {role:<15} {nom:<28} lat={lat:.4f}, lon={lon:.4f}")
52
53 print(f"\n      Matrice des distances (km) :")
54 header = f"{'':>22}" + "".join(f"{'C'+str(i):>8}" for i in range(N))
55 print(f"      {header}")
56 for i in range(N):
57     row = f"      {'C'+str(i)+' '+NOMS[i][:14]:>22}" + \
58         "".join(f"{'D[i][j]:>8.2f}" for j in range(N))

```

```

58         "".join(f"{'D[i][j]:>8.2f}" for j in range(N))
59     print(row)
60
61 # ÉTAPE 2 – Trouver le meilleur parcours (3 méthodes)
62 print(f"\n[2] Recherche du meilleur parcours...")
63
64 def longueur_route(route):
65     """Calcule la longueur totale d'un circuit (retour au dépôt inclus)."""
66     return sum(D[route[i]][route[i+1]] for i in range(len(route)-1))
67
68 autres = [i for i in range(N) if i != DEPOT]
69
70 # -- Méthode A : Force brute (exacte, optimale) ---
71 print(f"\n -- Méthode A : Force Brute (solution exacte) --")
72 best_dist_bf = float("inf")
73 best_route_bf = None
74
75 for perm in itertools.permutations(autres):
76     route = [DEPOT] + list(perm) + [DEPOT]
77     d = longueur_route(route)
78     if d < best_dist_bf:
79         best_dist_bf = d
80         best_route_bf = route
81
82 print(f"      Parcours : {' ' + '.join('C'+str(i) for i in best_route_bf)}")
83 print(f"      Noms      : {' ' + '.join(NOMS[i].split('(')[0].strip() for i in best_route_bf)}")
84 print(f"      Distance : {best_dist_bf:.3f} km")
85

```

```

87 # --- Méthode B : Plus proche voisin (greedy) ---
88 print("\n - Méthode B : Plus Proche Voisin (Greedy) -")
89 visite = [False] * N
90 route_pv = [DEPOT]
91 visite[DEPOT] = True
92 for _ in range(N - 1):
93     current = route_pv[-1]
94     prochain = min(
95         (j for j in range(N) if not visite[j]),
96         key=lambda j: D[current][j]
97     )
98     route_pv.append(prochain)
99     visite[prochain] = True
100 route_pv.append(DEPOT)
101 dist_pv = longueur_route(route_pv)
102
103 print(f"    Parcours : {' → '.join('C'+str(i) for i in route_pv)}")
104 print(f"    Noms    : {' → '.join(NOMS[i].split('(')[0].strip() for i in route_pv)}")
105 print(f"    Distance : {dist_pv:.3f} km")
106 print(f"    Écart vs optimal : +{dist_pv - best_dist_bf:.3f} km "
107       | f"+({(dist_pv/best_dist_bf-1)*100:.1f}%)")
108
109 # --- Méthode C : 2-opt (amélioration locale) ---
110 print("\n - Méthode C : 2-opt (amélioration locale) -")
111
112 def deux_opt(route):
113     """Améliore une route par échanges 2-opt jusqu'à convergence."""
114     ameliorer = True
115     while ameliorer:

```

```

116         ameliorer = False
117         for i in range(1, len(route) - 2):
118             for j in range(i + 1, len(route) - 1):
119                 nouvelle = route[:i] + route[i:j+1][::-1] + route[j+1:]
120                 if longueur_route(nouvelle) < longueur_route(route):
121                     route = nouvelle
122                     ameliorer = True
123     return route
124
125 route_2opt = deux_opt(route_pv[:]) # part de la solution greedy
126 dist_2opt = longueur_route(route_2opt)
127
128 print(f"    Parcours : {' → '.join('C'+str(i) for i in route_2opt)}")
129 print(f"    Noms    : {' → '.join(NOMS[i].split('(')[0].strip() for i in route_2opt)}")
130 print(f"    Distance : {dist_2opt:.3f} km")
131 print(f"    Écart vs optimal : +{dist_2opt - best_dist_bf:.3f} km "
132       | f"+({(dist_2opt/best_dist_bf-1)*100:.1f}%)")
133
134
135 # --- Méthode D : NetworkX TSP approximé ---
136 print("\n - Méthode D : NetworkX TSP (Christofides approché) -")
137 G_tsp = nx.Graph()
138 for i in range(N):
139     for j in range(i+1, N):
140         G_tsp.add_edge(NOMS[i], NOMS[j], weight=D[i][j])
141
142 route_nx_noms = nx.approximation.traveling_salesman_problem(
143     G_tsp, weight="weight", cycle=True

```

```

145 # Convertir noms → indices
146 nom_to_idx = {n: i for i, n in enumerate(NOMS)}
147 route_nx = [nom_to_idx[n] for n in route_nx_noms]
148 dist_nx = longueur_route(route_nx)
149
150 print(f"    Parcours : {' → '.join('C'+str(i) for i in route_nx)}")
151 print(f"    Noms    : {' → '.join(NOMS[i].split('(')[0].strip() for i in route_nx)}")
152 print(f"    Distance : {dist_nx:.3f} km")
153 print(f"    Écart vs optimal : +{dist_nx - best_dist_bf:.3f} km "
154       | f"+({(dist_nx/best_dist_bf-1)*100:.1f}%)")
155
156
157 # --- ÉTAPE 3 - Comparaison des méthodes ---
158 print("\n[3] Tableau comparatif des méthodes :")
159 print(f"\n    {'Méthode':<35} {'Distance (km)':>15} {'vs Optimal':>12} {'Complexité'}")
160 print(f"    {'-'*80}")
161
162 methodes = [
163     ("Force Brute (exact, optimal)",    best_dist_bf, "-",    "O(n!)"),
164     ("Plus Proche Voisin (Greedy)",     dist_pv,    dist_pv,    "O(n²)"),
165     ("2-opt (amélioration locale)",     dist_2opt,  dist_2opt, "O(n²).iter"),
166     ("NetworkX TSP (Christofides)",     dist_nx,    dist_nx,    "O(n³)"),
167 ]
168
169 for nom, dist, dist_cmp, complexite in methodes:
170     if dist_cmp == "-":
171         ecart = "    référence"
172     else:

```

```

172     else:
173         ecart = f"+{dist_cmp - best_dist_bf:.3f} km (+{(dist_cmp/best_dist_bf-1)*100:.1f}%"
174         print(f"      {nom:<35} {dist:>15.3f} {ecart:>20} {complexite}")
175
176 VITESSE = 20 # km/h tap-tap/moto
177 print(f"\n      Temps de livraison estimé (à {VITESSE} km/h) :")
178 for nom, dist, _, _ in methodes:
179     print(f"      {nom:<35} → {dist/VITESSE*60:.0f} min")
180
181 # --- Conseil final ---
182 print(f"""
183     Recommandation pratique :
184     • Pour un petit réseau (≤ 10 clients) : Force Brute (solution exacte)
185     • Pour un réseau moyen (10-100)       : 2-opt (bon compromis)
186     • Pour un grand réseau (> 100)         : Christofides / heuristiques
187     • Optimal trouvé : {best_dist_bf:.2f} km pour {N-1} clients
188 """)
189
190 # Visualisation comparative
191 pos = {NOMS[i]: (COORDS[i][1], COORDS[i][0]) for i in range(N)}
192 node_colors = [{"#EF4444" if i == DEPOT else "#3B82F6" for i in range(N)]
193
194 def draw_route(ax, route_idx, titre, couleur, dist_km):
195     ax.set_facecolor("#0F172A")
196     for i in range(len(route_idx) - 1):
197         u = NOMS[route_idx[i]]
198         v = NOMS[route_idx[i+1]]
199         xu, yu = pos[u]
200         xv, yv = pos[v]

```

```

194 def draw_route(ax, route_idx, titre, couleur, dist_km):
200     xv, yv = pos[v]
201     ax.annotate("",
202                 xy=(xv, yv), xytext=(xu, yu),
203                 arrowprops=dict(arrowstyle="→", color=couleur,
204                                 lw=2.2, mutation_scale=15))
205     for i, nom in enumerate(NOMS):
206         x, y = pos[nom]
207         couleur_n = "#EF4444" if i == DEPOT else "#60A5FA"
208         ax.scatter(x, y, s=200, color=couleur_n, zorder=5, edgecolors="white", linewidths=1.2)
209         label = nom.split("(")[0].strip()
210         ax.annotate(label, (x, y), textcoords="offset points",
211                     xytext=(5, 6), fontsize=7, color="white", fontweight="bold")
212         ax.set_title(f"{titre}\n{dist_km:.2f} km", color="white", fontsize=10, pad=8)
213         ax.axis("off")
214
215 fig, axes = plt.subplots(2, 2, figsize=(16, 12))
216 fig.patch.set_facecolor("#060B18")
217 fig.suptitle("TD5 - Comparaison des méthodes d'optimisation de livraison\n"
218             "(Port-au-Prince)",
219             color="white", fontsize=14, fontweight="bold")
220
221 configs = [
222     (axes[0, 0], best_route_bf, "Force Brute (optimal)", "#10B981", best_dist_bf),
223     (axes[0, 1], route_pv, "Greedy (+ proche voisin)", "#F59E0B", dist_pv),
224     (axes[1, 0], route_2opt, "2-opt (amélioration locale)", "#8B5CF6", dist_2opt),
225     (axes[1, 1], route_nx, "NetworkX TSP (Christofides)", "#EC4899", dist_nx),
226 ]
227

```

```

227
228 for ax, route, titre, couleur, dist in configs:
229     draw_route(ax, route, titre, couleur, dist)
230
231 # Légende commune
232 leg = [
233     mpatches.Patch(color="#EF4444", label="Dépôt"),
234     mpatches.Patch(color="#60A5FA", label="Point de livraison"),
235 ]
236 fig.legend(handles=leg, loc="lower center", ncol=2,
237            facecolor="#0F172A", edgecolor="#475569",
238            labelcolor="white", fontsize=11)
239
240 plt.tight_layout(rect=[0, 0.04, 1, 1])
241 plt.savefig("TD5_livraison_comparaison.png", dpi=150, bbox_inches="tight",
242            facecolor="#060B18")
243 print("      Graphique sauvegardé : TD5_livraison_comparaison.png")
244 plt.show()
245
246 print("\n" + "-" * 65)
247 print("-" * 65)
248

```

[1] Définition des points de livraison...
7 points définis (1 dépôt + 6 clients)

Points de livraison :

DÉPÔT	Dépôt (Delmas 18)	lat=18.5614, lon=-72.3182
Client 1	Client 1 (Pétion-V.)	lat=18.5133, lon=-72.2869
Client 2	Client 2 (Tabarre)	lat=18.5942, lon=-72.2906
Client 3	Client 3 (Carrefour)	lat=18.5344, lon=-72.3708
Client 4	Client 4 (Cité Sol.)	lat=18.5788, lon=-72.3464
Client 5	Client 5 (Delmas 33)	lat=18.5500, lon=-72.3100
Client 6	Client 6 (Croix-B.)	lat=18.5459, lon=-72.3465

Matrice des distances (km) :

	C0	C1	C2	C3	C4	C5	C6
C0 Dépôt (Delmas	0.00	6.28	4.67	6.31	3.55	1.53	3.45
C1 Client 1 (Péti	6.28	0.00	9.00	9.15	9.61	4.75	7.25
C2 Client 2 (Taba	4.67	9.00	0.00	10.76	6.13	5.32	7.97
C3 Client 3 (Carr	6.31	9.15	10.76	0.00	5.57	6.64	2.86
C4 Client 4 (Cité	3.55	9.61	6.13	5.57	0.00	5.00	3.66
C5 Client 5 (Delm	1.53	4.75	5.32	6.64	5.00	0.00	3.87
C6 Client 6 (Croï	3.45	7.25	7.97	2.86	3.66	3.87	0.00

[2] Recherche du meilleur parcours...

- Méthode A : Force Brute (solution exacte) -
Parcours : C0 → C2 → C4 → C6 → C3 → C1 → C5 → C0
Noms : Dépôt → Client 2 → Client 4 → Client 6 → Client 3 → Client 1 → Client 5 → Dépôt
Distance : 32.750 km
- Méthode B : Plus Proche Voisin (Greedy) -
Parcours : C0 → C5 → C6 → C3 → C4 → C2 → C1 → C0
Noms : Dépôt → Client 5 → Client 6 → Client 3 → Client 4 → Client 2 → Client 1 → Dépôt
Distance : 35.253 km
Écart vs optimal : +2.503 km (+7.6%)
- Méthode C : 2-opt (amélioration locale) -
Parcours : C0 → C2 → C4 → C6 → C3 → C1 → C5 → C0
Noms : Dépôt → Client 2 → Client 4 → Client 6 → Client 3 → Client 1 → Client 5 → Dépôt
Distance : 32.750 km
Écart vs optimal : +0.000 km (+0.0%)
- Méthode D : NetworkX TSP (Christofides approché) -
Parcours : C0 → C4 → C3 → C6 → C2 → C1 → C5 → C0
Noms : Dépôt → Client 4 → Client 3 → Client 6 → Client 2 → Client 1 → Client 5 → Dépôt
Distance : 35.240 km
Écart vs optimal : +2.490 km (+7.6%)

[3] Tableau comparatif des méthodes :

Méthode	Distance (km)	vs Optimal	Complexité
Force Brute (exact, optimal)	32.750		référence $O(n!)$
Plus Proche Voisin (Greedy)	35.253	+2.503 km (+7.6%)	$O(n^2)$
2-opt (amélioration locale)	32.750	+0.000 km (+0.0%)	$O(n^2) \cdot \text{iter}$
NetworkX TSP (Christofides)	35.240	+2.490 km (+7.6%)	$O(n^3)$

Temps de livraison estimé (à 20 km/h) :

Force Brute (exact, optimal)	→ 98 min
Plus Proche Voisin (Greedy)	→ 106 min
2-opt (amélioration locale)	→ 98 min
NetworkX TSP (Christofides)	→ 106 min

Recommandation pratique :

- Pour un petit réseau (≤ 10 clients) : Force Brute (solution exacte)
- Pour un réseau moyen (10-100) : 2-opt (bon compromis)
- Pour un grand réseau (> 100) : Christofides / heuristiques
- Optimal trouvé : 32.75 km pour 6 clients

Graphique sauvegardé : TD5_livraison_comparaison.png

